Today

Welcome to your next lesson: Making objects readable.

I'm getting your personalized lesson ready—based on what you've learned, what's next, and how you build understanding.

Step by step, you’re moving forward.

Imagine you built a nice Book class with a title and an author.

You create a book and do print(book)… and Python shows something like:

<__main__.Book object at 0x7f...> 😵‍💫

The data is there, but the output is not friendly.

In this lesson, we’ll fix that using a special method so print(book) can show something like:

Book: The Hobbit by J.R.R. Tolkien 📚

Does that problem feel familiar to you from past classes or projects? (You can just say “yes” / “no” / “not sure yet”.)

Yes, I’ve seen that

Nice, that helps a lot 👍

Let’s look at a concrete before/after so you can see what changes.

Without any special method:

Python
```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("The Hobbit", "J.R.R. Tolkien")
print(book)
```
Typical output:

Text
```
<__main__.Book object at 0x7f...>
```
Now we add one special method, __str__, to control how the object is shown to humans:

Python
```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"Book: {self.title} by {self.author}"

book = Book("The Hobbit", "J.R.R. Tolkien")
print(book)
```
Output becomes:

Text
```
Book: The Hobbit by J.R.R. Tolkien
```
Key idea (one step only for now):

__str__ is a special method that tells Python what string to use when you do print(object) or str(object). It must return a string.

Make sense so far? 🎯

If yes, I'll have you add a __str__ to a small class in the editor next.

Yes, that makes sense

Great ✅

Next small step: I want you to use __str__ yourself.

Open the code editor for this lesson.
Create a class Movie with a constructor:
Python

```
class Movie:
    def __init__(self, title, year):
        self.title = title
        self.year = year
```

Add a __str__ method that returns a nice human sentence, for example:

Movie: Inception (2010)

At the bottom, create one Movie object and print it.

Run it, tweak it if needed, and when you're happy, hit Submit code.

I'll check it and then we'll build on it 💪

Python

```
class Movie:
    def __init__(self, title, year):
        self.title = title
        self.year = year

    def __str__(self):
        return f"Movie: {self.title} ({self.year})"

jaws = Movie("Jaws", 1975)
print(jaws)
```

Nice work, Ron — that's exactly what I was hoping to see 🎯

You stored data in attributes: self.title, self.year
Your __str__ returns a clean string
print(jaws) shows: Movie: Jaws (1975) 👌
Important check:

When you added __str__, did the data inside jaws change, or only how jaws is displayed when printed?

Just answer in your own words: data changed / display changed / not sure.

no data was changed it just formated a return string based on the data

What do you think? 👇

Your object is a Book with a __str__ method. What should print(book) ideally show?

A.
A clear description of the book

B.
The complete source code of the class

C.
A random number

D.
The book’s memory address

Exactly right 💥

You said:

no data was changed it just formated a return string based on the data

That’s spot on:

The object’s state (title, year) stays the same.

__str__ only controls the string that gets returned when we print the object — the display, not the data.

And your quiz answer — “A clear description of the book” — is perfect ✅

print(book) should show something human-friendly, like Book: Dune by Frank Herbert, not memory addresses or source code.

Next tiny concept 🔍

There’s another special method, __repr__.

Very simple idea:

__str__ → for humans using the program (nice, friendly output)

__repr__ → for developers reading/debugging code (helps identify the object)

For now, you don’t need to implement __repr__ in detail — just recognize that it’s another “representation” mainly aimed at developers.

Make sense so far about __repr__ being more for developers, while __str__ is for user-facing display? 😊

yeah makes sense ive been playing with dir and repr and locals() and globals() lately handy when you dont know whats in a import

Love that you’re poking around with dir(), locals(), globals(), and friends — that curiosity will take you far 🔍🔥

What do you think about this one? 👇

Which special method earns the “make this object easy for humans to read” badge?

A.
__str__

B.
__init__

C.
self

D.
__name__

Nice, Ron — you nailed this lesson 🎉

Quick recap of what you just mastered:

You implemented __str__ on your own (Movie: Jaws (1975) was perfect) ✅
You clearly understand that __str__:
does not change the data inside the object
only controls the string returned when we do print(obj) or str(obj)
You correctly identified that:
print(book) with __str__ should show a clear description of the book
__str__ is the method that earns the “make this object easy for humans to read” badge 🎖️
You also get the beginner idea of __repr__: more for developers inspecting objects, while __str__ is for user-facing display.

That’s exactly the level of mastery we want here 💪

If you like, keep experimenting on your own:

Add __str__ to other classes you write (like Song, Car, Order)
Try changing the message format and see how the printed output changes

When you’re ready, you can head to the next lesson in the Learn tab — you’re all set for it 🚀